INFORME TÉCNICO: Propuesta de Arquitectura y Flujo de Datos para SGRI-UNICAFAM

Fecha: 27 de marzo de 2026

Rol: Diseño de flujo entre programas y datos

---

1. Resumen Ejecutivo

Este documento define la arquitectura técnica y el flujo de datos para el **Sistema de Gestión de la Investigación Institucional (SGRI-UNICAFAM)**, considerando la retroalimentación recibida.

El despliegue se realiza en **Microsoft Azure** para garantizar escalabilidad, seguridad y facilidad de mantenimiento.

---

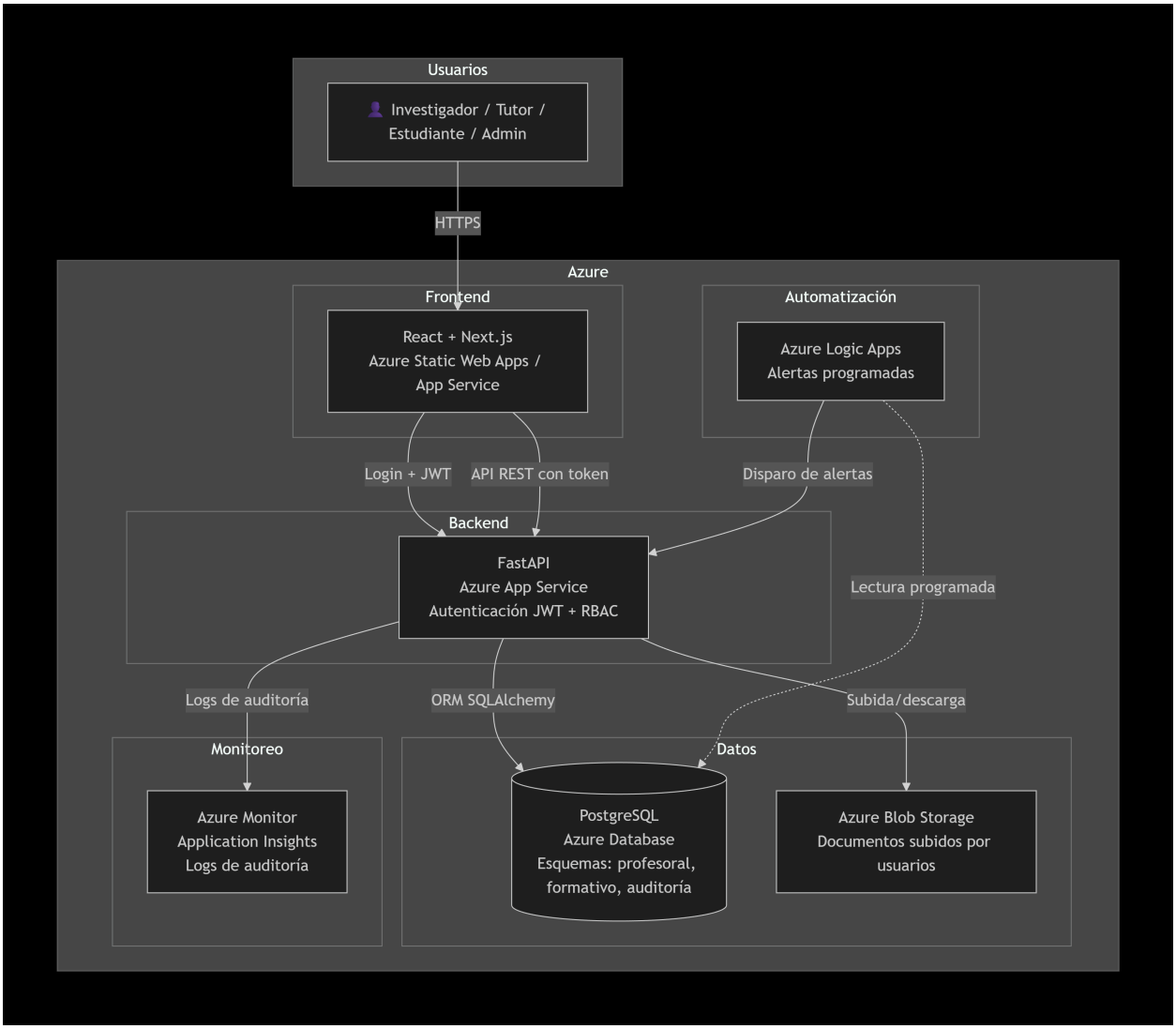
2. Alineación con los Requerimientos (MVP Final)

Requerimiento del SGRI	Implementación Propuesta (MVP)
Dato – Dueño – Calidad	Entidades con UUID, trazabilidad y roles responsables. Validación automática y por pares.
Módulos Profesorales (7) y Formativos (6)	Ambos frentes gestionados en la misma plataforma, con esquemas separados en la base de datos.
Roles RBAC (7 roles)	Control de acceso basado en roles implementado en el backend mediante lógica propia (JWT + permisos).
Indicadores y Dashboards	Cada proyecto (“repositorio”) muestra su propia barra de avance, métricas de cumplimiento y contribución a indicadores institucionales.
Alertas Automáticas	Sistema de tareas programadas (Azure Logic Apps) que notifica deadlines e inactividad.
Observabilidad y Auditoría	Registro completo de cambios (tablas de auditoría) y logs centralizados.

Requerimiento del SGRI	Implementación Propuesta (MVP)
Integraciones Externas	**Postergadas para fases posteriores**: Minciencias, CNA, RedCOLSI.
Exportaciones	**Fuera del alcance del MVP**: no se generarán reportes Excel, PDF ni certificados.

3. Arquitectura Técnica de Alto Nivel (MVP)

3.1 Diagrama de Arquitectura (Mermaid)



3.2 Descripción de Componentes

Componente	Función	Azure Servicio Sugerido
Frontend	Interfaz React + Next.js. Dashboards, formularios, visualización de métricas.	Azure Static Web Apps o App Service
Backend API	FastAPI. Lógica de negocio, autenticación JWT, RBAC, validaciones, cálculo de indicadores, despacho de alertas.	Azure App Service (Linux, Python)
Base de Datos	PostgreSQL con esquemas separados por dominio (profesoral, formativo, auditoría).	Azure Database for PostgreSQL – Flexible Server
Almacenamiento	Documentos subidos por usuarios (actas, avales, productos). No se generan reportes exportables.	Azure Blob Storage
Automatización	Tareas programadas: alertas de deadlines, inactividad de semilleros.	Azure Logic Apps (o Azure Functions)
Monitoreo	Logs de auditoría, métricas de rendimiento.	Azure Monitor, Application Insights

---

4. Flujo de Datos (Ejemplo: Creación de un Proyecto)

1. **Usuario** inicia sesión enviando credenciales a `/auth/token`. FastAPI valida y devuelve un token JWT.
2. **Frontend** almacena el token y lo envía en cada petición (cabecera `Authorization: Bearer <token>`).
3. Usuario envía datos del nuevo proyecto a `/api/proyectos`. El backend valida el token y los permisos del rol.
4. **Backend** inserta el proyecto en la tabla `proyectos` (esquema `profesoral`) y registra la acción en `auditoria`.
5. Si hay archivos adjuntos, se almacenan en **Blob Storage** y se guarda la URL.
6. El backend responde con éxito y el frontend actualiza la vista.
7. **Azure Logic Apps** ejecuta una rutina diaria: consulta proyectos próximos a vencer y envía correos automáticos a los responsables.

-

5. Roadmap Ajustado (Énfasis en MVP)

Fase	Actividades	Componentes Implicados
1 – Fundamentos	Modelo de datos, autenticación JWT, CRUD de facultades, investigadores, semilleros.	PostgreSQL, FastAPI, frontend básico.
2 – Núcleo Operativo	Módulos de semilleros, proyectos formativos, gestión de eventos RedCOLSI, indicadores de cobertura.	Ampliación de esquemas, endpoints específicos.
3 – Observabilidad	Dashboards en tiempo real, alertas automáticas.	Logic Apps, mejoras frontend.
4 – Integración y Madurez	(Futuro) Integración con CNA, RedCOLSI, módulo de presupuesto, app móvil.	Conectores externos, optimización.

6. Beneficios de la Arquitectura Ajustada

- **Autenticación simplificada:** Sin dependencia de servicios externos; el backend maneja JWT.
- **Modularidad:** Los componentes en Azure permiten incorporar nuevas capacidades más adelante sin reestructurar.
- **Costo eficiente:** Uso de servicios PaaS con pago por consumo.